Java Threads

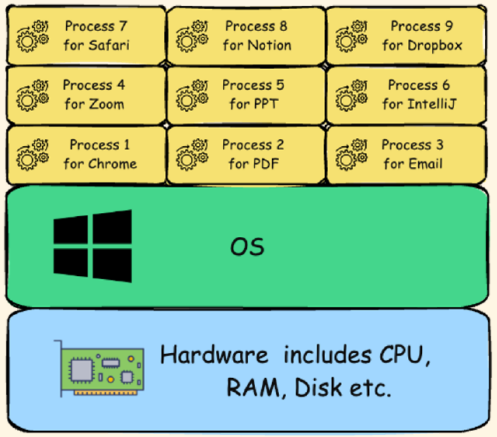
Section Overview



Multithreading in Java refers to the concurrent execution of multiple threads within a Java program. A thread is a lightweight sub-process, and multithreading allows you to perform multiple tasks simultaneously, improving the overall efficiency of your program.

Section Overview

- How a program or a software executes inside a computer?



The relationship between hardware, the operating system (OS), processes, and threads is crucial for the execution of programs on a computer. Here's an overview:

Hardware:
The hardware consists of the physical components of a computer, such as the CPU, memory, storage, input/output devices, etc. The CPU is the key component responsible for executing instructions.

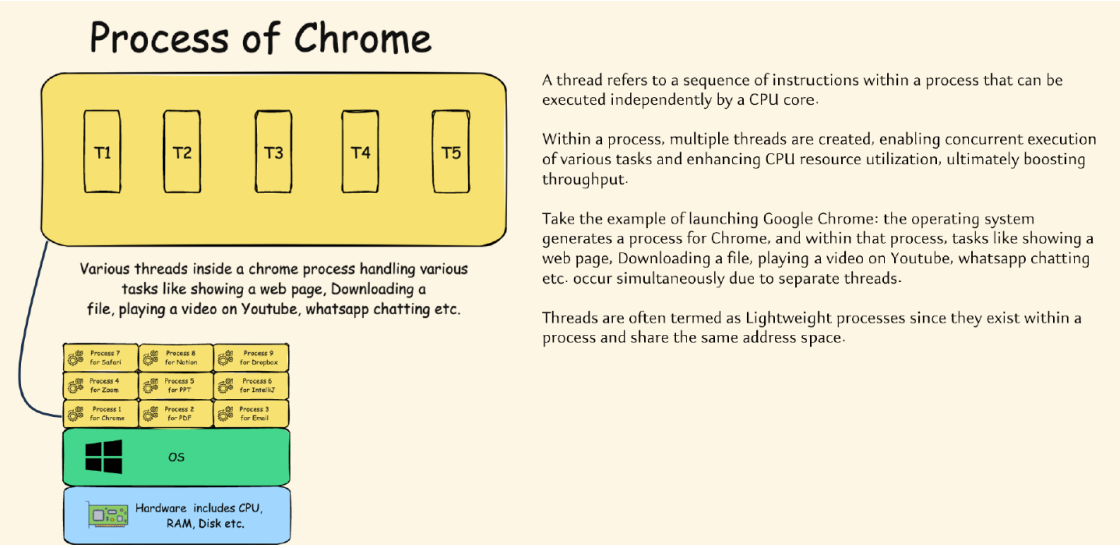
Operating System (OS):
The OS is system software that manages hardware resources and provides services for computer programs. It abstracts the hardware, providing a consistent interface for applications to run on different types of hardware.

Processes:
A process is a program in execution. It is an independent unit that runs in its own memory space. The OS manages processes, allocates resources, and provides isolation between them. Each process has its own address space, file descriptors, and system resources.

Threads:
A thread is the smallest unit of execution within a process. Multiple threads can exist within a single process. Threads within a process share the same memory space, allowing for efficient communication.Threads can run concurrently, and the OS schedules their execution on the CPU.

Section Overview

- How a program or a software executes inside a computer?

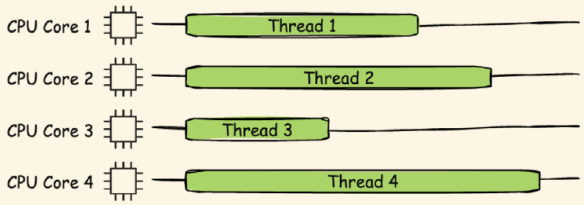


Section Overview

- Parallel vs Concurrent Execution

Parallel Execution

In the realm of computing, parallel execution involves a computer simultaneously performing multiple tasks. Consider a scenario with a 4-core CPU where each core is engaged in running a distinct task, allowing for genuine simultaneous execution of all four tasks.



Analogy: Imagine a well-organized kitchen with four chefs, each working on a separate dish. Each chef independently prepares their dish at the same time. As a result, all four dishes are ready parallely.

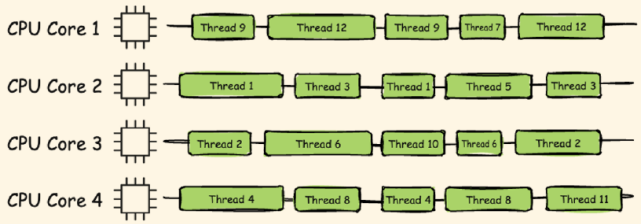


Section Overview

- Parallel vs Concurrent Execution

Concurrent Execution

On the other hand, concurrent execution creates the appearance of simultaneous task execution when the number of tasks exceeds the available CPU cores. For instance, with a 4-core CPU, attempting to execute 12 different tasks triggers the operating system to engage in context-switching. This process gives the illusion of concurrently handling all eight tasks, even though, in reality, only four instructions can execute at any given time due to the limitation of having four cores.





Analogy: Now, envision the same kitchen, but with only four cooking stations (cores) available. However, there are 12 dishes to be prepared. In this case, the chefs (tasks) take turns using the available cooking stations. While it may seem like all 12 dishes are being prepared at once, only four chefs can actively cook at any given time due to the limited number of cooking stations.

Section Overview

- How multiple threads improve performance?

With a CPU having just 4 cores, it implies that at most 4 parallel executions can occur simultaneously. Each thread must be assigned to a CPU core for execution, given the 4-core limitation, resulting in the capability to run only 4 threads in parallel.

However, a question arises: why do applications often incorporate hundreds of threads? What purpose does this serve? Why not limit the number of threads to match the CPU core count?

Let's explore the reasons behind this practice below.

Tasks can be categorized into two main types: CPU-bound and IO-bound.



CPU-Bound tasks are those whose execution heavily relies on the CPU, involving operations like arithmetic, logical processing, and relational calculations.



On the other hand, IO-Bound tasks are highly dependent on Input/Output operations, such as communication with a network or performing read/write operations on a file within a file system.



Consider a scenario that exclusively involves CPU tasks, such as a mathematical computation algorithm. For instance, a program might perform intricate mathematical calculations, analyze large datasets, and generate complex statistical models—all without engaging in any input/output operations. In this context, the tasks are purely CPU-bound, emphasizing the computational capabilities of the system.

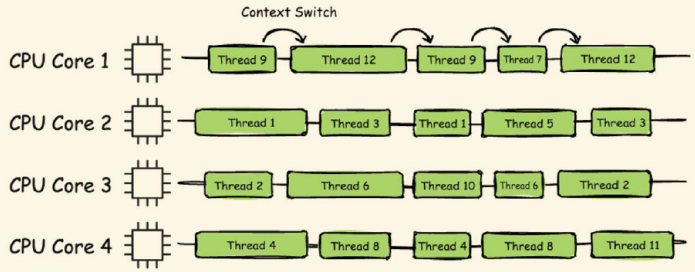


In practice, tasks often involve a combination of both CPU and IO operations. Take, for example, a task where data is fetched from a database (IO), complex calculations are performed on the retrieved data (CPU), and the results are then sent to a remote server (IO). Here, the database interaction and server communication involve IO operations, while the computational processing falls under CPU-bound activity. This illustrates the typical scenario of tasks combining various types of operations.

Section Overview

- How multiple threads improve performance?

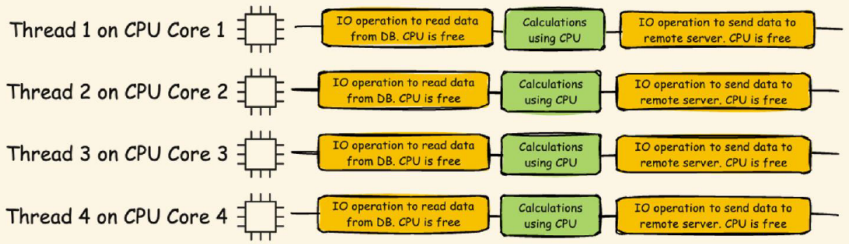
Examining our CPU-intensive arithmetic task executed across 12 threads reveals that, despite the potential for concurrent execution, the actual benefit is questionable. The CPU, with its 4 cores, engages in frequent context-switching between these threads before any of them completes. In this context, employing 12 threads is counter productive, as it leads to inefficient utilization of CPU resources. For tasks primarily bound by CPU processing, it is more beneficial to align the number of threads closely with the available cores to optimize efficiency.



Section Overview

- How multiple threads improve performance?

Now let's take the example of the task where data is fetched from a database (IO), complex calculations are performed on the retrieved data (CPU), and the results are then sent to a remote server (IO). First let's try running on only 4 threads since 4 cores are available.

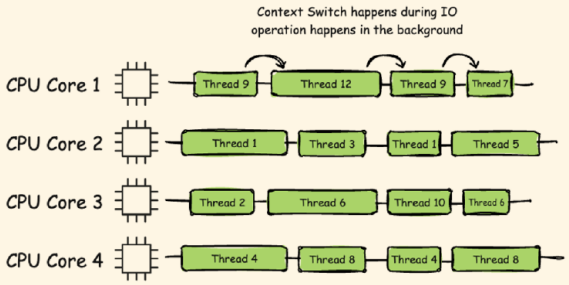


When we have IO operations involved and choose exactly same number of Threads as CPU cores, we will end up wasting lot of CPU power as CPU sits idle during the IO operations like shown above.

Section Overview

- How multiple threads improve performance?

Let's see what happens if we run the same task on 12 threads.

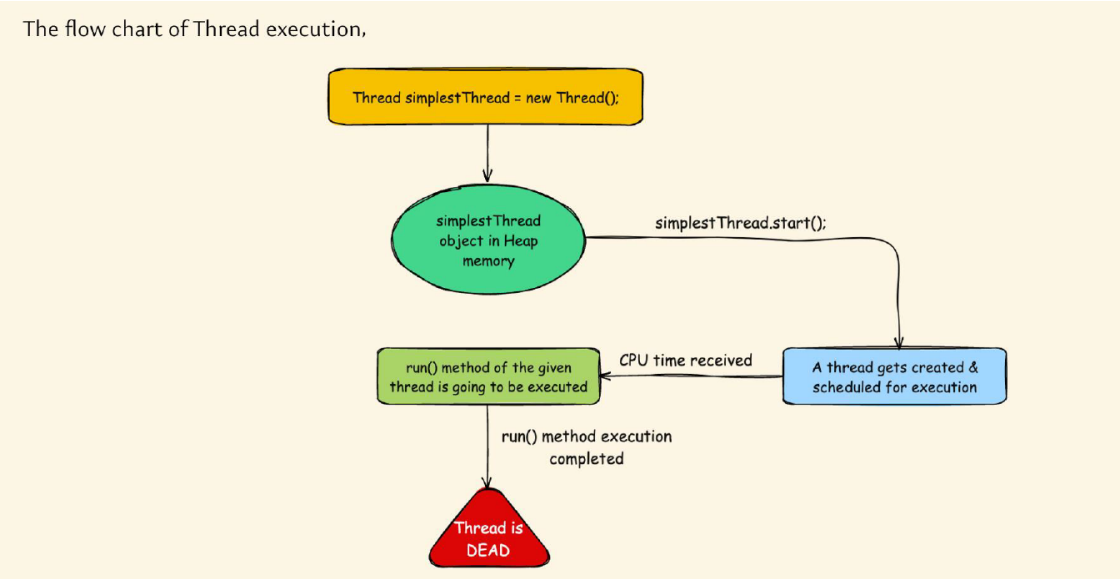


Focusing on the first core, it is clear that as Thread 9 executes, it may encounter an I/O operation causing the CPU to go idle. However, with multiple threads at hand, instead of remaining idle, the CPU swiftly switches to another thread and continues executing it until Thread 9 completes the I/O operation. This way, the full capacity of the CPU can be effectively utilized, optimizing performance with multiple threads.

In our initial arithmetic calculation only task, using more than 4 threads proved counterproductive due to unnecessary context switching, given its highly CPU-bound nature. However, in the current scenario with 12 threads, efficiency is improved. This underscores the importance of judiciously choosing the number of threads based on the frequency and duration of IO operations. The optimal number of threads is directly linked to the quantity of IO operations in the system. Higher IO demands warrant a greater number of threads to enhance efficiency.

Section Overview

- Flowchart of Thread Execution



Creating Threads in Java

- Three approaches to specify code for execution by a thread.
- 1. By deriving your class from the thread class
- 2.By implementing the runnable interface
- 3.Using lambda expression

Example program by using Thread class

```
> class MyThread extends Thread { public void run() { System.out.println("Thread is running"); } } public class Main { public static void main(String[] args) { MyThread t = new MyThread(); t.start(); } }
```

Section Overview

```
> public class AnotherThread extends Thread{ public void run() //Overriding run method of Thread class { System.out.println("Hello from another thread"); } }
```

```
> public class Main { public static void main(String[] args) { System.out.println("Hello from main thread"); Thread anotherThread = new AnotherThread(); anotherThread.start(); //To invoke run method System.out.println("Hello from main again"); } }
```

- Example program by using Thread class
- Output:

```
Hello from main thread
Hello from main again
Hello from another thread
```

The order of execution of thread depends on the JVM and OS.

- Each execution may produce different order of the result.
- Main Thread:
- It is the thread from where other “child” thread are spawned
- It must be the last thread to finish execution.
- When the main thread stops, the program terminates.
- Main thread can also be controlled using a Thread object

Creating thread by implementing Runnable interface

```
> public class MyRunnable implements Runnable{ @Override public void run() { System.out.println("Hello from My Runnable implementation"); } }
```

```
> public class Main { public static void main(String[] args) { System.out.println("Hello from main thread"); Thread myRunnable = new Thread(new MyRunnable()); myRunnable.start(); } }
```

Using Lambda Expressions

- We can use lambda expressions to instantiate the ‘Runnable’ interface.

```
> public class Demo {
> public static void main(String[] args) {
```

- Thread t = new Thread(() -> {

```
> System.out.println("Thread is running");
```

- for (int i = 1; i <= 5; i++) {

```
> System.out.println(i);
```

- }
- });
- t.start();
- }
- }

Section Overview

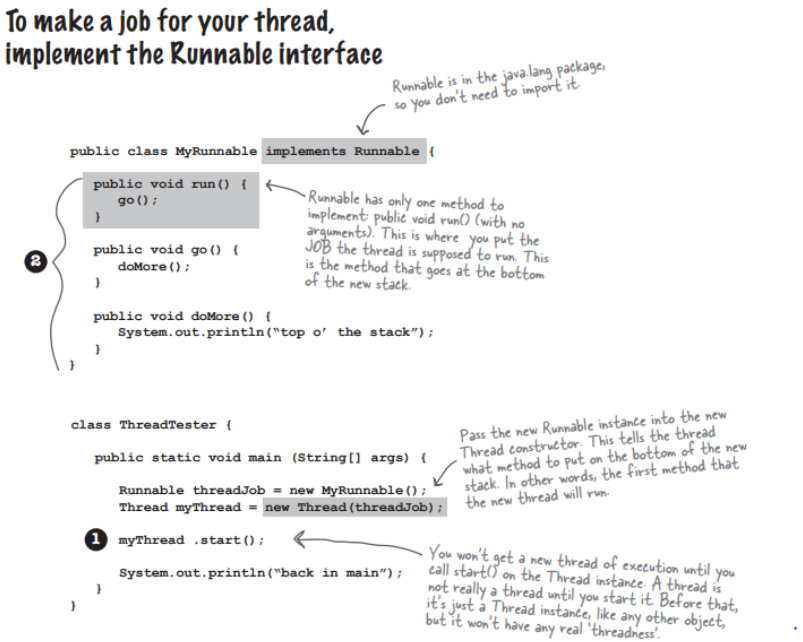
For added conciseness, we can employ a method reference, either static or instance, that takes no parameters and returns void as the code to be executed by a thread. The subsequent code introduces a Hello class featuring an sayHello() method, encapsulating the code intended for execution within a thread.

```
public class Hello {
    public static void sayHello() {
        System.out.println("Hello from Java threads!");
    }
}
```

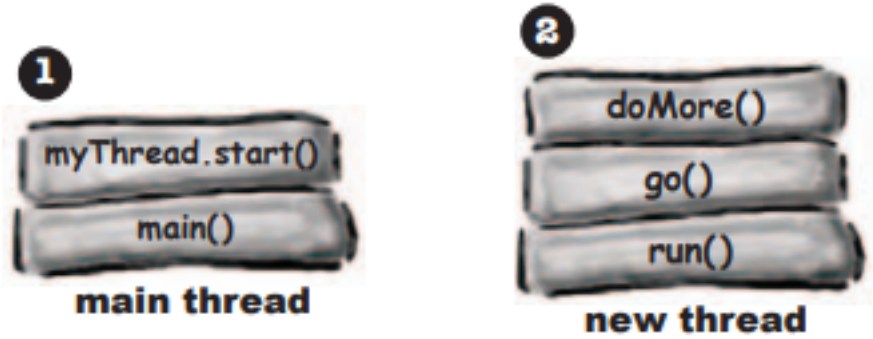
Below code uses the method reference of the sayHello() method of the Hello class to create a Runnable object:

```
Thread methRefThread = new Thread(Hello::sayHello);
methRefThread.start();
```

Section Overview



Section Overview



Section Overview

Making and starting *two* threads

Threads have names. You can give your threads a name of your choosing, or you can accept their default names. But the cool thing about names is that you can use them to tell which thread is running. The following example starts two threads. Each thread has the same job: run in a loop, printing the currently-running thread's name with each iteration.

```
public class RunThreads implements Runnable {

    public static void main(String[] args) {
        RunThreads runner = new RunThreads();
        Thread alpha = new Thread(runner);
        Thread beta = new Thread(runner);
        alpha.setName("Alpha thread");
        beta.setName("Beta thread");
        alpha.start();
        beta.start();
    }

    public void run() {
        for (int i = 0; i < 25; i++) {
            String threadName = Thread.currentThread().getName();
            System.out.println(threadName + " is running");
        }
    }
}
```

Make one Runnable instance.

Make two threads, with the same Runnable (the same job--we'll talk more about the "two threads and one Runnable" in a few pages).

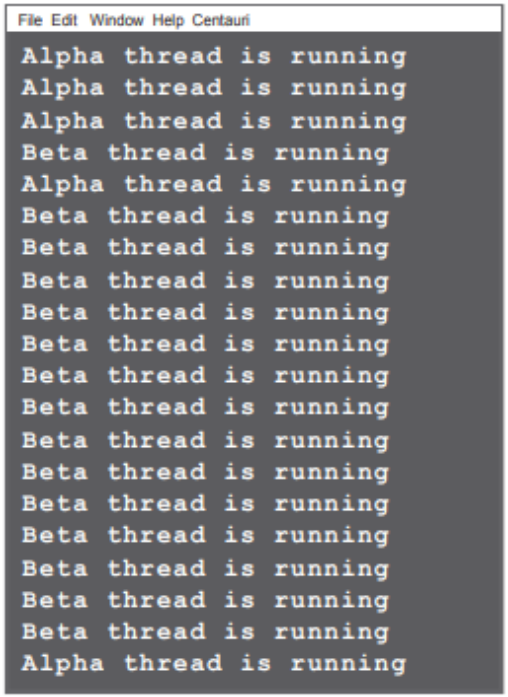
Name the threads.

Start the threads.

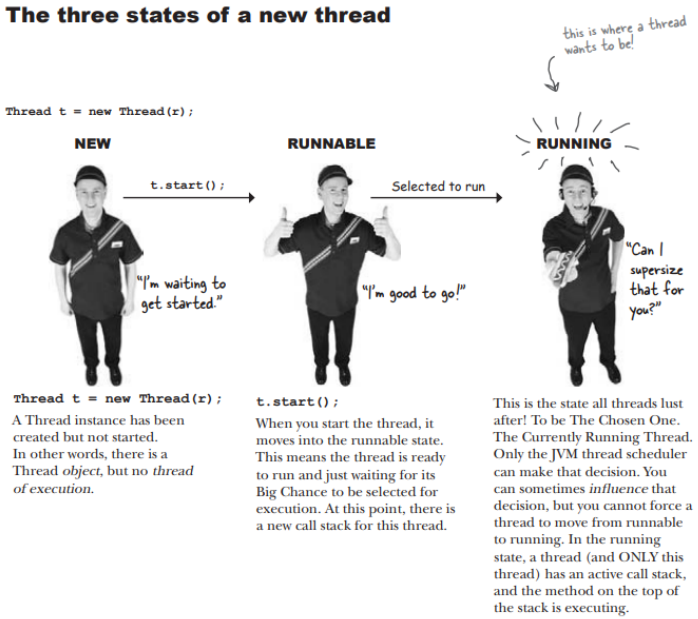
Each thread will run through this loop, printing its name each time.

Section Overview

- Output:



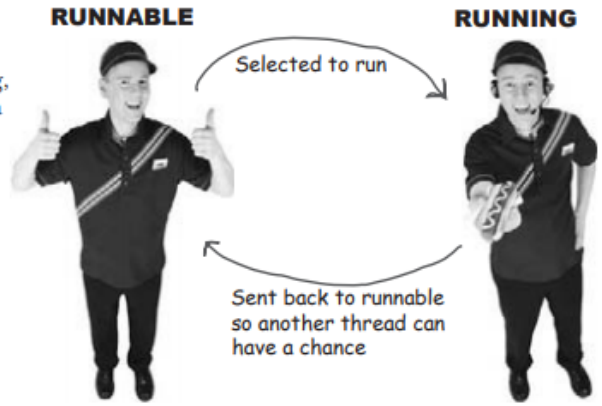
Section Overview



Section Overview

Typical runnable/running loop

Typically, a thread moves back and forth between runnable and running, as the JVM thread scheduler selects a thread to run and then kicks it back out so another thread gets a chance.

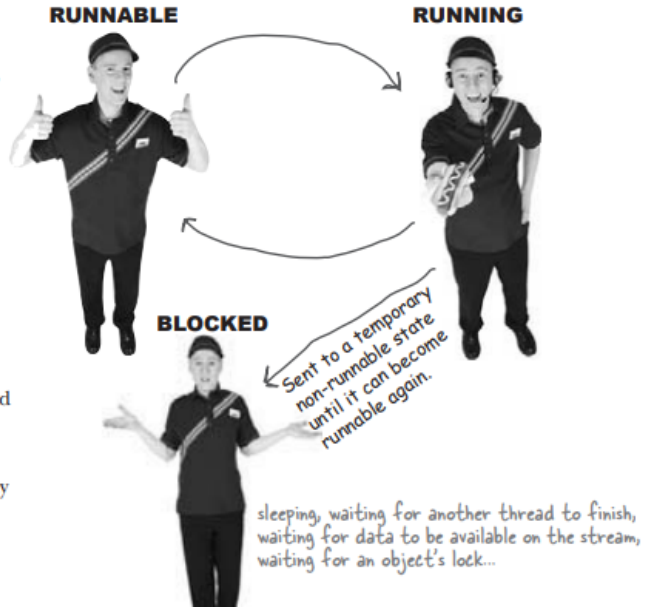


Section Overview

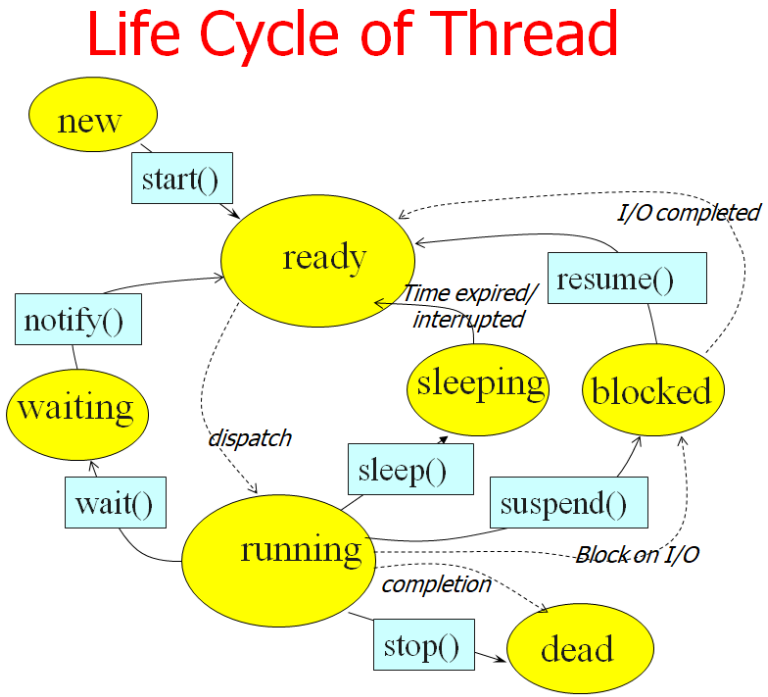
A thread can be made temporarily not-runnable

The thread scheduler can move a running thread into a blocked state, for a variety of reasons. For example, the thread might be executing code to read from a Socket input stream, but there isn't any data to read. The scheduler will move the thread out of the running state until something becomes available. Or the executing code might have told the thread to put itself to sleep (sleep()). Or the thread might be waiting because it tried to call a method on an object, and that object was 'locked'. In that case, the thread can't continue until the object's lock is freed by the thread that has it.

All of those conditions (and more) cause a thread to become temporarily not-runnable.



Section Overview



The life cycle of threads in Java is very similar to the life cycle of processes running in an operating system.

During its life cycle the thread moves from one state to another depending on the operation performed by it or performed on it.

- A Java thread can be in one of the following states:

NEW A thread that is just instantiated is in new state. When a start() method is invoked, the thread moves to the ready state from which it is automatically moved to runnable state by the thread scheduler.

- RUNNABLE (ready running) A thread executing in the JVM is in running state.

BLOCKED A thread that is blocked waiting for a monitor lock is in this state. This can also occur when a thread performs an I/O operation and moves to next (runnable) state.

- THREAD LIFECYCLE

WAITING A thread that is waiting indefinitely for another thread to perform a particular action is in this state.

TIMED_WAITING (sleeping) A thread that is waiting for another thread to perform an action for up to a specified waiting time is in this state.

- TERMINATED (dead) A thread that has exited is in this state.

Section Overview

An example of how unpredictable the scheduler can be...

Running this code on one machine:

```
public class MyRunnable implements Runnable {

   public void run() {
      go();
   }

   public void go() {
      doMore();
   }

   public void doMore() {
      System.out.println("top o' the stack");
   }
}


class ThreadTestDrive {

   public static void main (String[] args) {

      Runnable threadJob = new MyRunnable();
      Thread myThread = new Thread(threadJob);

      myThread.start();

      System.out.println("back in main");
   }
}
```

Notice how the order changes randomly. Sometimes the new thread finishes first, and sometimes the main thread finishes first.

Produced this output:

```
File Edit Window Help PickMe
% java ThreadTestDrive
back in main
top o' the stack
% java ThreadTestDrive
top o' the stack
back in main
% java ThreadTestDrive
top o' the stack
back in main
% java ThreadTestDrive
top o' the stack
back in main
% java ThreadTestDrive
top o' the stack
back in main
% java ThreadTestDrive
top o' the stack
back in main
% java ThreadTestDrive
top o' the stack
back in main
% java ThreadTestDrive
back in main
top o' the stack
```

Section Overview

- Using sleep to make our program more predictable.

Remember our earlier example that kept giving us different results each time we ran it? Look back and study the code and the sample output. Sometimes main had to wait until the new thread finished (and printed "top o' the stack"), while other times the new thread would be sent back to runnable before it was finished, allowing the main thread to come back in and print out "back in main". How can we fix that? Stop for a moment and answer this question: "Where can you put a sleep() call, to make sure that "back in main" always prints before "top o' the stack"?

We'll wait while you work out an answer (there's more than one answer that would work).

Section Overview

```
public class MyRunnable implements Runnable {

   public void run() {
      go();
   }

   public void go() {

      try {
        Thread.sleep(2000);
      } catch(InterruptedException ex) {
        ex.printStackTrace();
      }

      doMore();
   }

   public void doMore() {
      System.out.println("top o' the stack");
   }
}

class ThreadTestDrive {
   public static void main (String[] args) {
      Runnable theJob = new MyRunnable();
      Thread t = new Thread(theJob);
      t.start();
      System.out.println("back in main");
   }
}
```

- Calling sleep here will force the new thread to leave the currently-running state!

The main thread will become the currently-running thread again, and print out "back in main". Then there will be a pause (for about two seconds) before we get to this line, which calls doMore() and prints out "top o' the stack"

Section Overview



sleep()

Thread.sleep() method can be used to pause the execution of current thread for specified time in milliseconds. The argument value for milliseconds can’t be negative, else it throws IllegalArgumentException.

Thread.sleep() interacts with the thread scheduler to put the current thread in wait state for specified period of time. Once the wait time is over, thread state is changed to runnable state and wait for the CPU for further execution. So the actual time that current thread sleep depends on the thread scheduler that is part of operating system

- sleep() should be always written within a try catch block as it will throw an InterruptedException

Thread Priority

In Java, each thread is assigned priority, which affects the order in which it is scheduled for running. The threads so far had same default priority (NORM_PRIORITY) and they are served using FCFS policy.

- Java allows users to change priority:
  - ThreadName.setPriority(intNumber)
  - MIN_PRIORITY = 1
  - NORM_PRIORITY=5
  - MAX_PRIORITY=10

class X extends Thread

- {
- public void run()
- {

```
> System.out.println("Thread x Started");
```

- for(int i=0; i<5; i++)
- {

```
> System.out.println("\t value of i in Thread x : " + i);
```

- }

```
> System.out.println("Threadx finished ");
```

- }
- }
- class Y extends Thread
- {
- public void run()
- {

```
> System.out.println("Thread Y started");
```

- for(int i=0; i<5; i++)
- {

```
> System.out.println("\tValue of i in Thread Y : " + i);
```

- }

```
> System.out.println("ThreadY Finished");
```

- }
- }

- Thread Priority Example

class Z extends Thread

- {
- public void run()
- {

```
> System.out.println("Thread Z started");
```

- for(int i=0; i<5; i++)
- {

```
> System.out.println("\tValue of i in Thread Z : " + i);
```

- }

```
> System.out.println("ThreadZ Finished");
```

- }
- }
- class ThreadPriority
- {
- public static void main(String[] args)
- {

```
> System.out.println("Main thread started");
```

- X threadX = new X();
- Y threadY = new Y ();
- Z threadZ = new Z ();
- threadZ.setPriority(Thread.MAX_PRIORITY); // priority = 10
- threadY.setPriority(threadX.getPriority()+1); // priority = 6
- threadX.setPriority(Thread.MIN_PRIORITY); // priority =l
- threadX.start(); threadY.start(); threadZ.start();

- try
- {
- threadX.join();threadY.join();threadZ.join();
- }
- catch(InterruptedException e){}

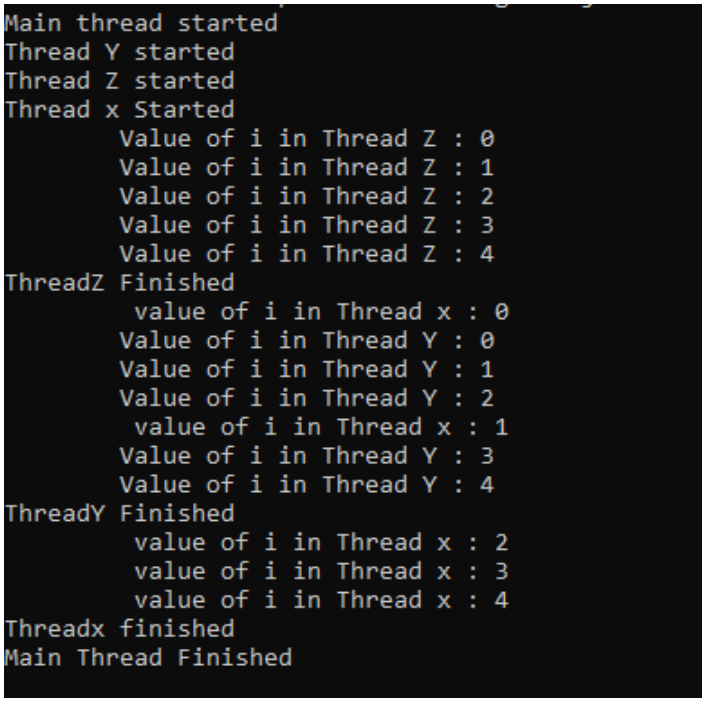
```
> System.out.println("Main Thread Finished");
```

- }
- }

Generally higher priority threads can be expected to be given preference by the thread scheduler over lower priority threads. However, the implementation of thread scheduling is left up to the JVM implementation.

- Output:

Although the threads are started in the order thread X followed by thread Y followed by thread Z but the thread Z (priority = 10) produces the output before thread Y (priority = 5) and thread X (priority = 1) because it has the highest priority.



Using isAlive( ) and join( )

Two ways exist to determine whether a thread has finished. First, you can call isAlive( ) on the thread. This method is defined by Thread, and its general form is shown here:

- final boolean isAlive( )

The isAlive( ) method returns true if the thread upon which it is called is still running. It returns false otherwise. While isAlive( ) is occasionally useful, the method that you will more commonly use to wait for a thread to finish is called join( ), shown here:

- final void join( ) throws InterruptedException
- This method waits until the thread on which it is called terminates.
- Its name comes from the concept of the calling thread waiting until the specified thread joins it.

Additional forms of join( ) allow you to specify a maximum amount of time that you want to wait for the specified thread to terminate.

Example of using join()

```
> public class JoinExample {
> public static void main(String[] args) {
```

- Thread th1 = new Thread(new MyClass(), "th1");
- Thread th2 = new Thread(new MyClass(), "th2");
- Thread th3 = new Thread(new MyClass(), "th3");

```
> // Start first thread immediately
```

- th1.start();
- /* Start second thread (th2) once first thread (th1)
- * is dead
- */
- try {
- th1.join();
- } catch (InterruptedException ie) {
- ie.printStackTrace();
- }
- th2.start();
- /* Start third thread (th3) once second thread (th2)
- * is dead
- */
- try {
- th2.join();
- } catch (InterruptedException ie) {
- ie.printStackTrace();

- }

Example of using join()

- th3.start();

```
> // Displaying a message once third thread is dead
```

- try {
- th3.join();
- } catch (InterruptedException ie) {
- ie.printStackTrace();
- }

```
> System.out.println("All three threads have finished execution");
```

- }
- }
- class MyClass implements Runnable {
- @Override
- public void run() {
- Thread t = Thread.currentThread();

```
> System.out.println("Thread started: " + t.getName());
```

- try {
- Thread.sleep(4000);
- } catch (InterruptedException ie) {
- ie.printStackTrace();
- }

```
> System.out.println("Thread ended: " + t.getName());
```

- }
- }

Section Overview

Output:

```
Thread started: th1
Thread ended: th1
Thread started: th2
Thread ended: th2
Thread started: th3
Thread ended: th3
All three threads have finished execution
```

Synchronization

When two or more threads need access to a shared resource, they need some way to ensure that the resource will be used by only one thread at a time. The process by which this is achieved is called synchronization. As you will see, Java provides unique, language-level support for it.

- Key to synchronization is the concept of the monitor.
- A monitor is an object that is used as a mutually exclusive lock.
- Only one thread can own a monitor at a given time.
- When a thread acquires a lock, it is said to have entered the monitor.
- All other threads attempting to enter the locked monitor will be suspended until the first thread exits the monitor.

These other threads are said to be waiting for the monitor. A thread that owns a monitor can reenter the same monitor if it so desires.

Understanding the problem without Synchronization

```
> class First { public void display(String msg) { System.out.print ("["+msg); try { Thread.sleep(500); } catch(InterruptedException e) { e.printStackTrace(); } System.out.println ("]"); } }
```

Understanding the problem without Synchronization

```
> class Second extends Thread { String msg; First fobj; Second (First fp,String str) { fobj = fp; msg = str; start(); } public void run() { fobj.display(msg); } } public class Main { public static void main (String[] args) { First fnew = new First(); Second ss = new Second(fnew, "welcome"); Second ss1 = new Second (fnew,"to"); Second ss2 = new Second(fnew, “java"); } }
```

- Output: (The order varies during each execution)

```
[java[welcome[to]
]
]

[java[to[welcome]
]
]
```

Using Synchronized Methods

- If you declare any method as synchronized, it is known as synchronized method.
- Synchronized method is used to lock an object for any shared resource.

When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the method returns.

- In the previous example include the keyword “synchronized” before the method “display”

class First { public synchronized void display(String msg) { System.out.print ("["+msg); try { Thread.sleep(500); } catch(InterruptedException e) { e.printStackTrace(); } System.out.println ("]"); } }

- The output will be :

```
[welcome]
[java]
[to]
```

The synchronized Statement

While creating synchronized methods within classes that you create is an easy and effective means of achieving synchronization, it will not work in all cases.

- Imagine that you want to synchronize access to objects of a class that was not designed for multithreaded access.
- That is, the class does not use synchronized methods.
- Further, this class was not created by you, but by a third party, and you do not have access to the source code.
- Thus, you can’t add synchronized to the appropriate methods within the class.

Fortunately, the solution to this problem is quite easy: You simply put calls to the methods defined by this class inside a synchronized block. This is the general form of the synchronized statement:

The synchronized Statement

Here, objRef is a reference to the object being synchronized. A synchronized block ensures that a call to a synchronized method that is a member of objRef's class occurs only after the current thread has successfully entered objRef's monitor.

```
synchronized(objRef) {
  // statements to be synchronized
}
```

// This program uses a synchronized block.

- class Callme {
- void call(String msg) {
- System.out.print("[" + msg);
- try {
- Thread.sleep(1000);
- } catch (InterruptedException e) {

```
> System.out.println("Interrupted");
```

- }

```
> System.out.println("]");
```

- }
- }
- class Caller implements Runnable {
- String msg;
- Callme target;
- Thread t;
- public Caller(Callme targ, String s) {
- target = targ;
- msg = s;
- t = new Thread(this);
- }

```
> // synchronize calls to call()
```

- public void run() {
- synchronized (target) { // synchronized block

- target.call(msg);
- }
- }
- }

Section Overview

- class Synch1 {

```
> public static void main(String args[]) {
```

- Callme target = new Callme();
- Caller ob1 = new Caller(target, "Hello");
- Caller ob2 = new Caller(target, "Synchronized");
- Caller ob3 = new Caller(target, "World");

```
> // Start the threads.
```

- ob1.t.start();
- ob2.t.start();
- ob3.t.start();

```
> // wait for threads to end
```

- try {
- ob1.t.join();
- ob2.t.join();
- ob3.t.join();
- } catch (InterruptedException e) {

```
> System.out.println("Interrupted");
```

- }
- }
- }

Interthread Communication

To avoid polling, Java includes an elegant interprocess communication mechanism via the wait( ), notify( ), and notifyAll( ) methods.

- These methods are implemented as final methods in Object, so all classes have them.
- All three methods can be called only from within a synchronized context.

Although conceptually advanced from a computer science perspective, the rules for using these methods are actually quite simple:

wait( ) tells the calling thread to give up the monitor and go to sleep until some other thread enters the same monitor and calls notify( ) or notifyAll( ). • notify( ) wakes up a thread that called wait( ) on the same object.

• notifyAll( ) wakes up all the threads that called wait( ) on the same object. One of the threads will be granted access

These methods are declared within Object, as shown here:

- final void wait( ) throws InterruptedException
- final void notify( )
- final void notify All( )

Although wait( ) normally waits until notify( ) or notifyAll( ) is called, there is a possibility that in very rare cases the waiting thread could be awakened due to a spurious wakeup

To begin, consider the following sample program that incorrectly implements a simple form of the producer/consumer problem.

- It consists of four classes:
- Q, the queue that you’re trying to synchronize;
- Producer, the threaded object that is producing queue entries;
- Consumer, the threaded object that is consuming queue entries;
- and
- PC, the tiny class that creates the single Q, Producer, and Consumer.

// A correct implementation of a producer and consumer.

- class Q {
- int n;
- boolean valueSet = false;
- synchronized int get() {
- while (!valueSet) {
- try {
- wait();
- } catch (InterruptedException e) {

```
> System.out.println("InterruptedException caught");
```

- }
- }

```
> System.out.println("Got: " + n);
```

- valueSet = false;
- notify();

```
> return n;
```

- }
- synchronized void put(int n) {
- while (valueSet) {
- try {
- wait();
- } catch (InterruptedException e) {

```
> System.out.println("InterruptedException caught");
```

- }

- }

this.n = n;

- valueSet = true;

```
> System.out.println("Put: " + n);
```

- notify();
- }
- }
- class Producer implements Runnable {
- Q q;
- Thread t;
- Producer(Q q) {
- this.q = q;
- t = new Thread(this, "Producer");
- }
- public void run() {
- int i = 0;
- while (true) {
- q.put(i++);
- }
- }
- }

Section Overview

- class Consumer implements Runnable {
- Q q;
- Thread t;
- Consumer(Q q) {
- this.q = q;
- t = new Thread(this, "Consumer");
- }
- public void run() {
- while (true) {
- q.get();
- }
- }
- }
- class PCFixed {

```
> public static void main(String args[]) {
```

- Q q = new Q();
- Producer p = new Producer(q);
- Consumer c = new Consumer(q);

```
> // Start the threads.
```

- p.t.start();
- c.t.start();

```
> System.out.println("Press Control-C to stop.");
```

- }

- }

Inside get( ), wait( ) is called.

- This causes its execution to suspend until Producer notifies you that some data is ready.
- When this happens, execution inside get( ) resumes. After the data has been obtained, get( ) calls notify( ).
- This tells Producer that it is okay to put more data in the queue.
- Inside put( ), wait( ) suspends execution until Consumer has removed the item from the queue.

When execution resumes, the next item of data is put in the queue, and notify( ) is called. This tells Consumer that it should now remove it.

- Output:

```
Put: 1
Got: 1
Put: 2
Got: 2
Put: 3
Got: 3
Put: 4
Got: 4
Put: 5
Got: 5
```

Obtaining a Thread’s State

- You can obtain the current state of a thread by calling the getState( ) method defined by Thread. It is shown here:
- Given a Thread instance, you can use getState( ) to obtain the state of a thread.

For example, the following sequence determines if a thread called thrd is in the RUNNABLE state at the time getState( ) is called:

- Thread.State ts = thrd.getState();
- if(ts == Thread.State.RUNNABLE) //

Depending on the circumstances, the state obtained by calling getState( ) may not reflect the actual state of the thread only a moment later.

Thread.State getState( )

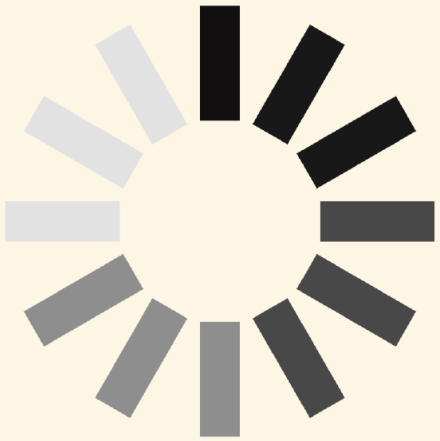
Suspending, Resuming, and Stopping Threads

Prior to Java 2, a program used suspend( ), resume( ), and stop( ), which are methods defined by Thread, to pause, restart, and stop the execution of a thread.

The suspend( ) method of the Thread class was deprecated by Java 2 several years ago. This was done because suspend( ) can sometimes cause serious system failures.

- The resume() and stop() methods were also deprecated.

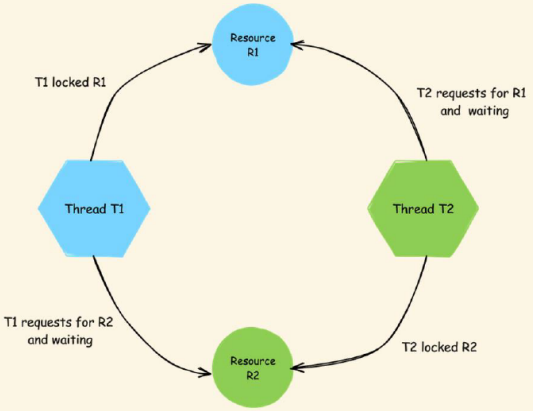
Section Overview



Deadlocks primarily occur in multithreaded programming scenarios. For instance, a deadlock may arise when one thread is waiting to acquire the lock of an object currently held by another thread, and simultaneously, the second thread is waiting to obtain the lock of a different object held by the first thread. Consequently, both threads find themselves in a state of mutual waiting, each expecting the other to release the lock, leading to a deadlock situation.

Section Overview

Imagine two threads, T1 and T2, each currently holding a specific resource – T1 has R1, and T2 has R2. Now, T1 is in the process of requesting resource R2, while simultaneously, T2 is requesting resource R1. Both threads are unwilling to release their existing resources, R1 and R2, until they complete their execution. This situation sets the stage for a deadlock, as illustrated in the image below.



Section Overview

```java
public class DeadLockDemo {

    public static final String R1 = "Hello";
    public static final String R2 = "Hi";

    public static void main(String[] args) {
        // Thread 1
        Thread thread1 = new Thread(() -> {
            synchronized (R1) { // R1 locked by T1
                System.out.println("Thread T1 locked : Resource R1");
                synchronized (R2) { // R2 locked by T1
                    System.out.println("Thread T1 locked : Resource R2");
                }}});

        // Thread 2
        Thread thread2 = new Thread(() -> {
            synchronized (R2) { // R2 locked by T2
                System.out.println("Thread T2 locked : Resource R2");
                synchronized (R1) { // R1 locked by T2
                    System.out.println("Thread T2 locked : Resource R1");
                }}});

        // starting both the threads
        thread1.start();
        thread2.start();
    }

}
```

Section Overview

- How to Avoid Deadlocks

Avoid Nested Locks

One of the most important reasons for deadlock in Java is nested locks. So if a thread already has a lock, try not to acquire another nested lock.

Avoid Unnecessary Locks

Only apply locks to the members that truly require them. Unnecessarily using locks on non-critical sections can also contribute to the likelihood of encountering a deadlock.

Lock Hierarchy:

Establish a hierarchy for acquiring locks and ensure that all threads follow this hierarchy. This approach helps prevent circular waiting scenarios.

Methods in Thread Class

The Thread class defines several methods that help manage threads. Several of those used in this chapter are shown here:

Method	Meaning
getName	Obtain a thread's name.
getPriority	Obtain a thread's priority.
isAlive	Determine if a thread is still running.
join	Wait for a thread to terminate.
run	Entry point for the thread.
sleep	Suspend a thread for a period of time.
start	Start a thread by calling its run method.